UAH Mars Drone ASW
Architecture Design Document

Index

5 Software Design

5.1 Software Static Architecture

5.2 Software Dynamic Architecture

5.2.1 Computational Model

5.2.2 Software Behaviour

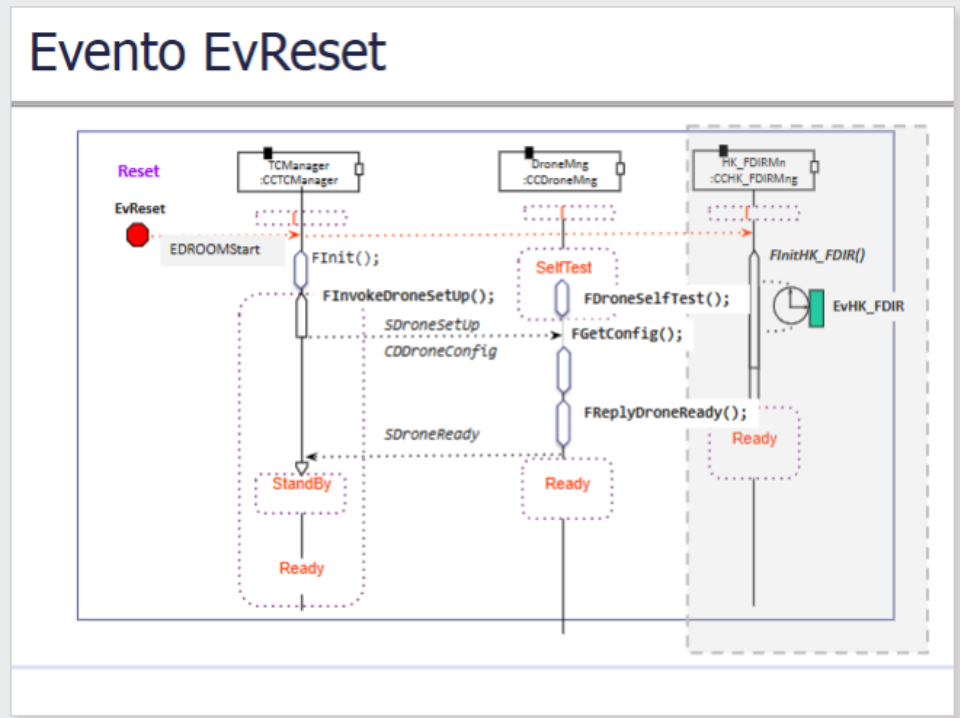
Scenarios

Event	Description
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptation
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
EvInitFlightPlan (interno)	El Drone Mng activa el temporizador DroneTimer asignándole un ciclo periódico de 100 ms
EvDroneTC (interno)	El sistema procesan los telecomandos de vuelo y se ajustan los parámetros PID (TC[129,1]), la configuración de la ruta TC[129,2], y la orden de ejecución del plan de vuelo TC[3,31]
EvReset (externo)	El sistema se inicializa y se ejecutan las acciones de self test y setup

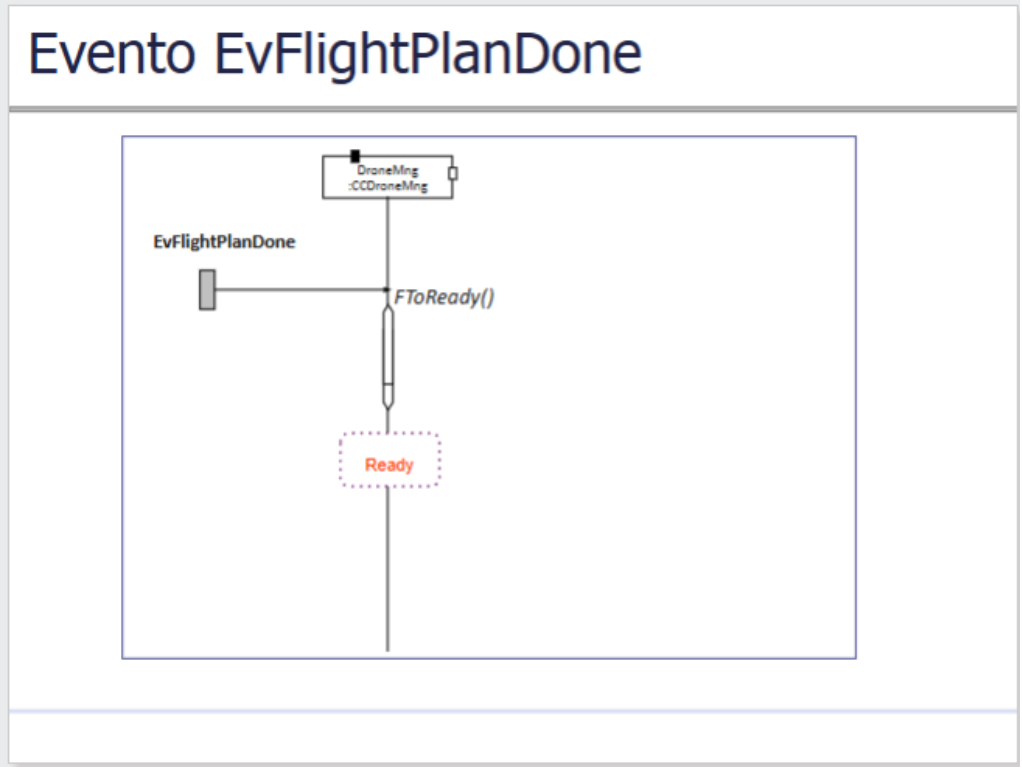
EvFlightCtrl (timing)	El componente reajusta continuamente el temporizador hasta que se acabe la ruta planificada
EvFlightPlanDone (internal)	Es el evento que indica que la ruta de vuelo ha concluido con éxito, lo que provoca que el DroneMng retorne a su estado de espera

TODO 02 Añade los diagramas de secuencia de aquellos escenarios ***en los que participa el componente DroneMng***

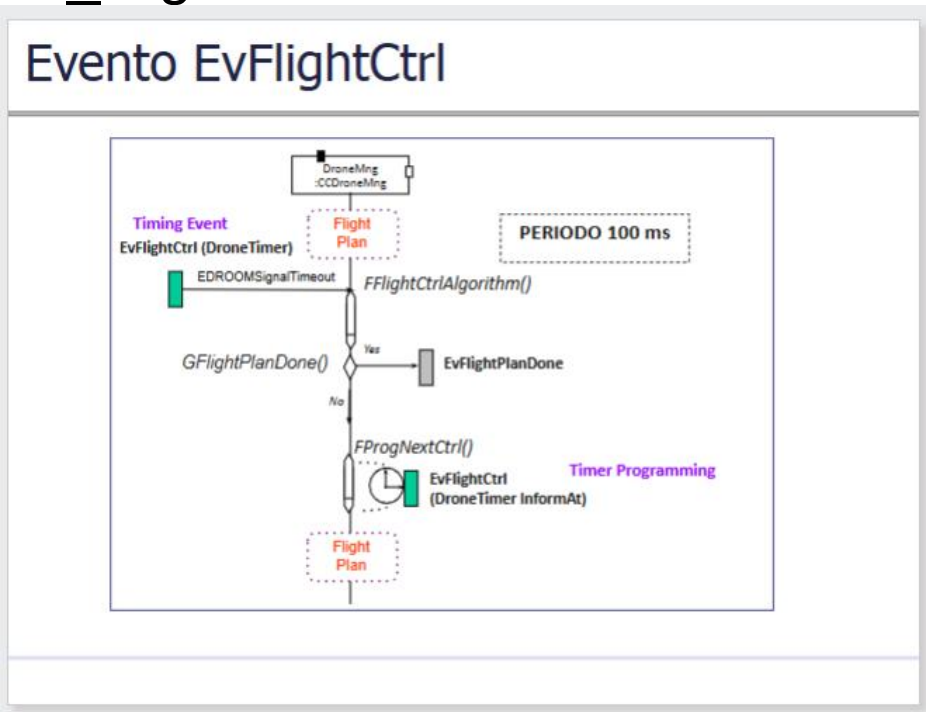
Ev_reset



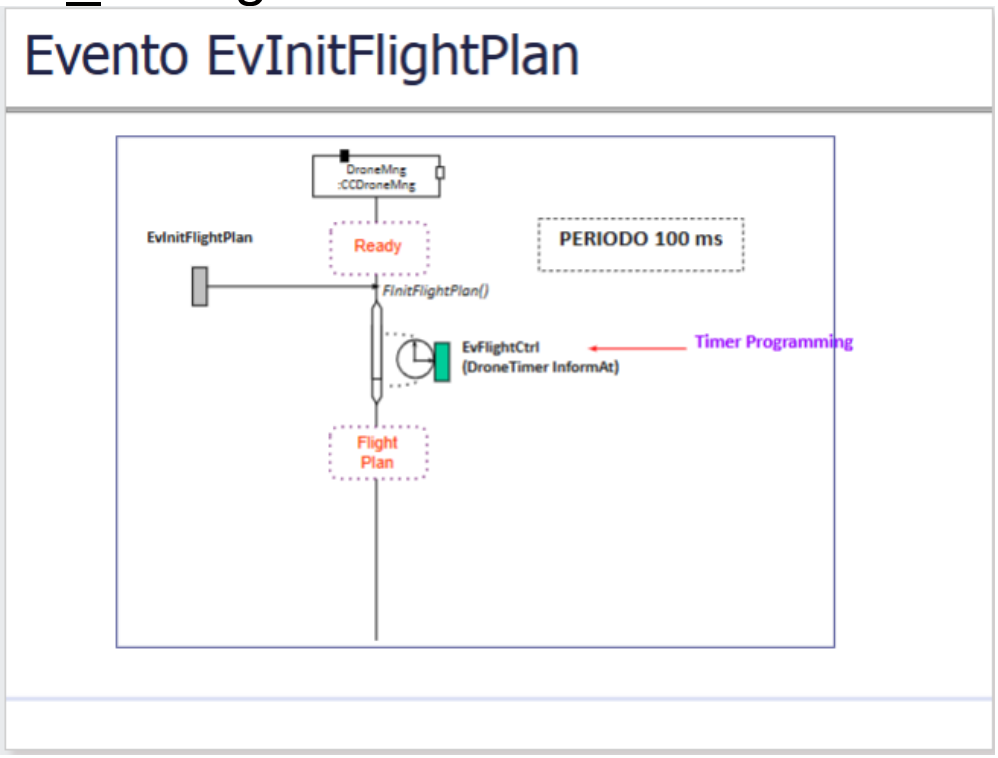
Ev_FlightPlanDone



Ev_FlightCtrl

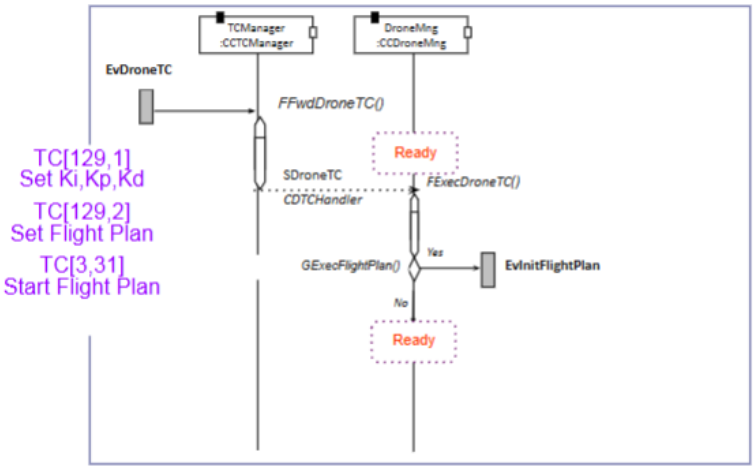


Ev_InitFlightPlan



Ev_DroneTC

Evento EvDroneTC



5.3 Real-Time Requirements

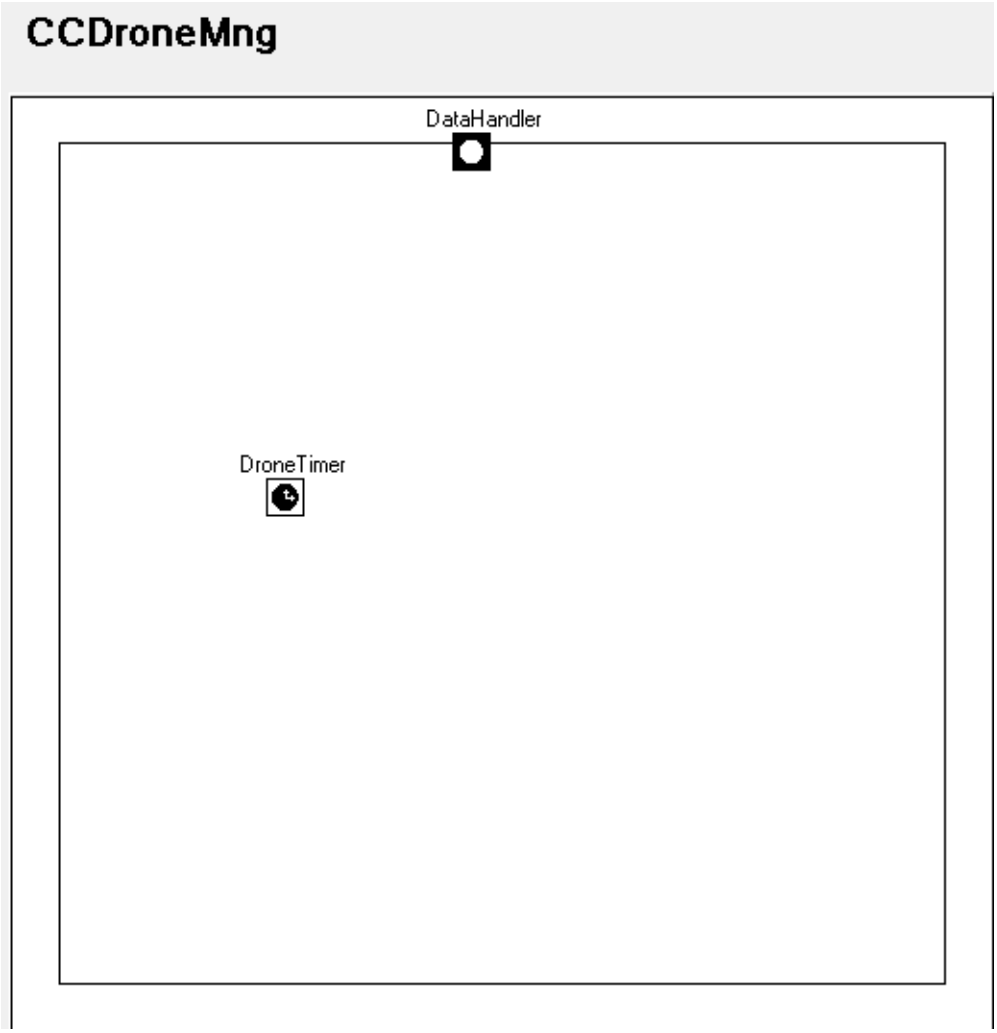
5.4 Software Components Design

5.4.1 CCDroneMng

5.4.1.1 Interfaces

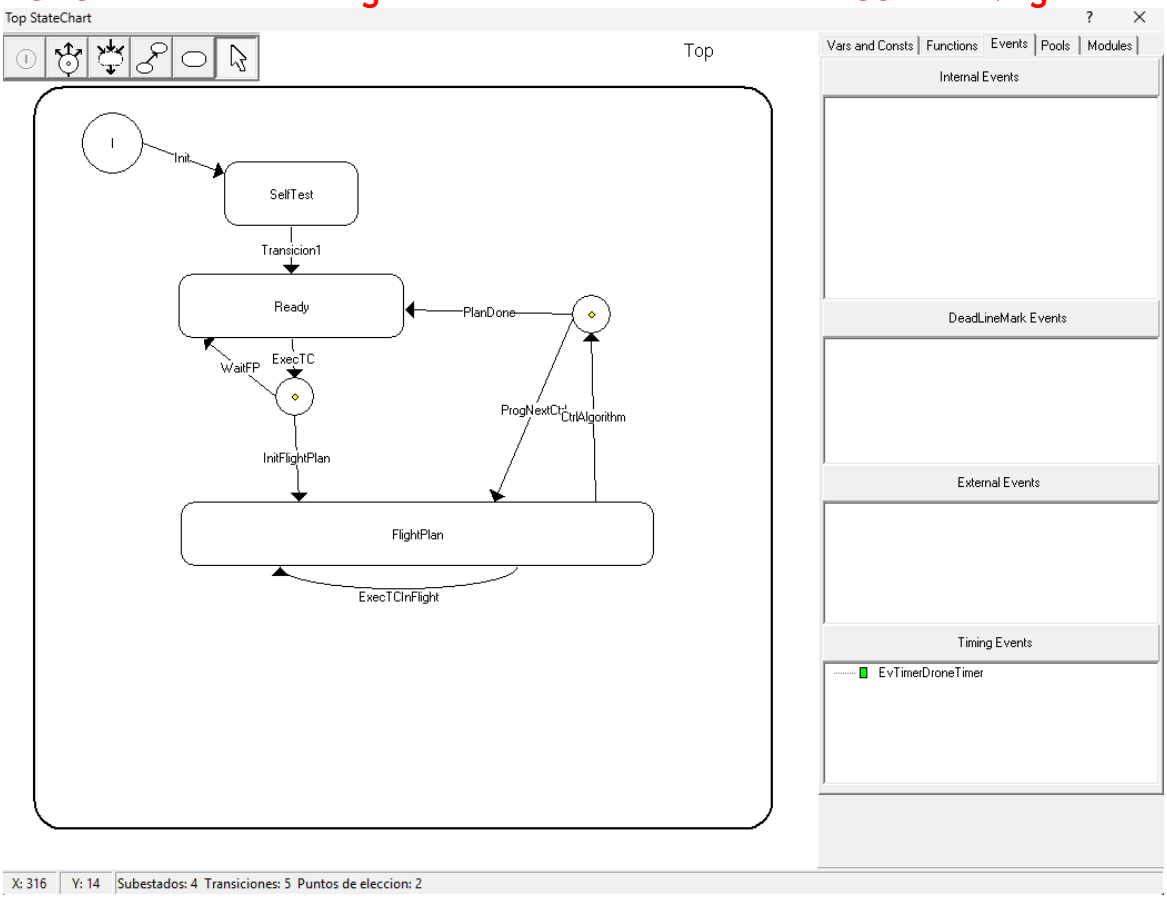
*TODO 03 Añade **los puertos que definen las interfaces de la clase CCDroneMng***

CCDroneMng **Interfaces**

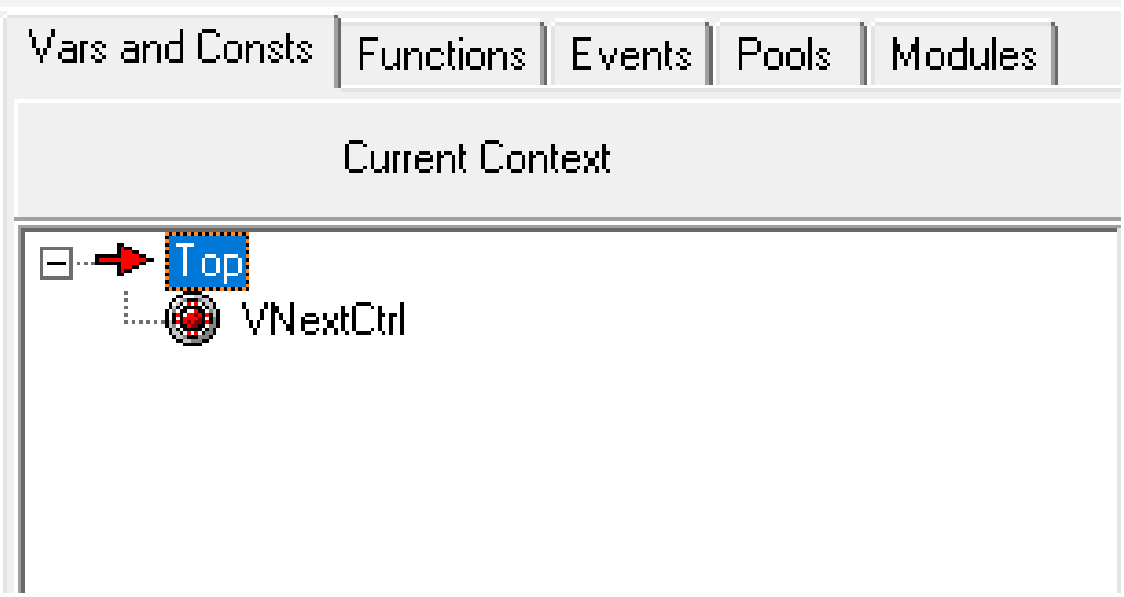


5.4.1.1 Behaviour

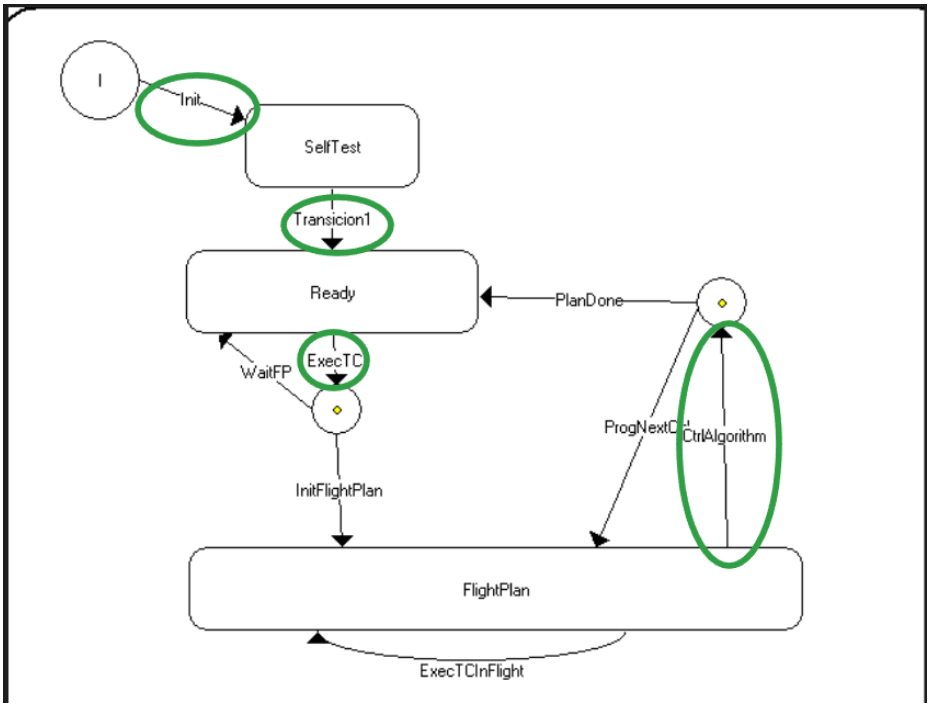
TODO 04 Añade el diagrama de estados de la clase CCDroneMng



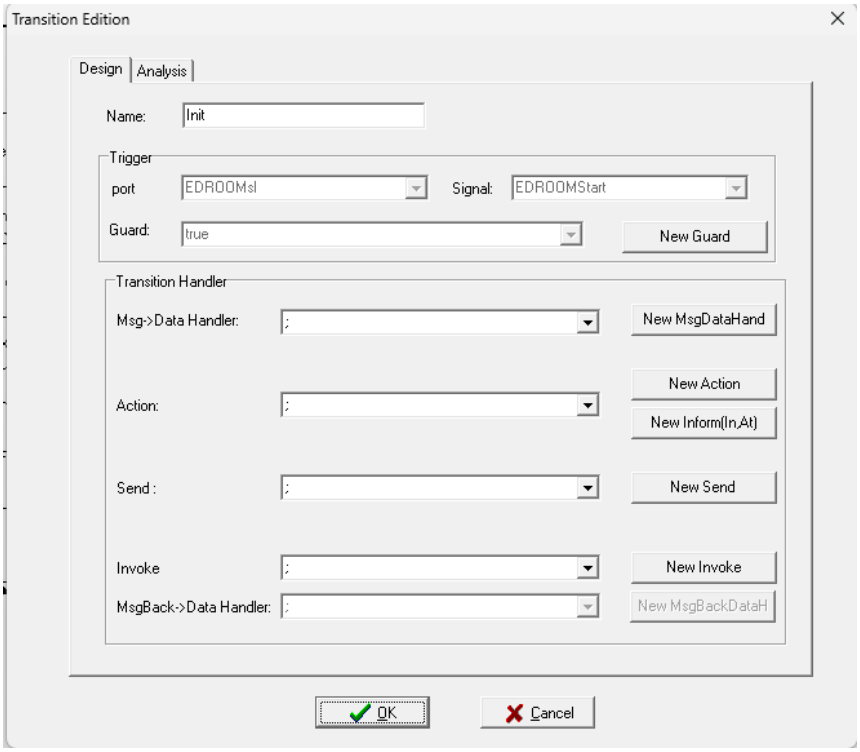
TODO 05 Añade las variables de la clase CCDroneMng



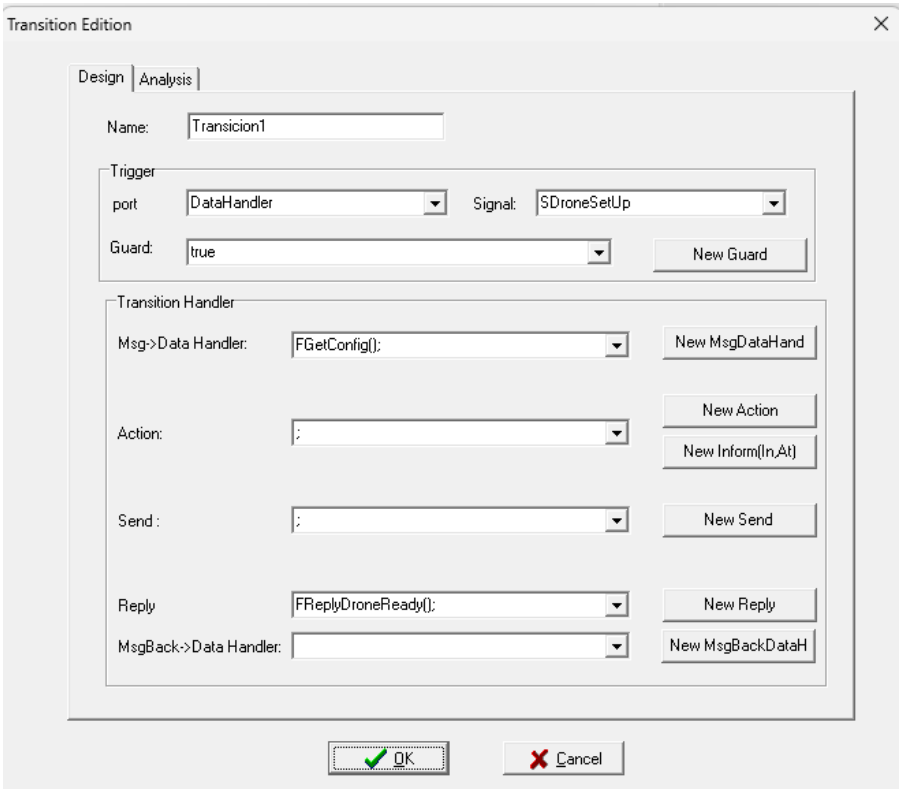
TODO 06 Añade la condición de disparo y la acción de cada una de las transiciones (utiliza el diagrama de estados varias veces si lo ves necesario)



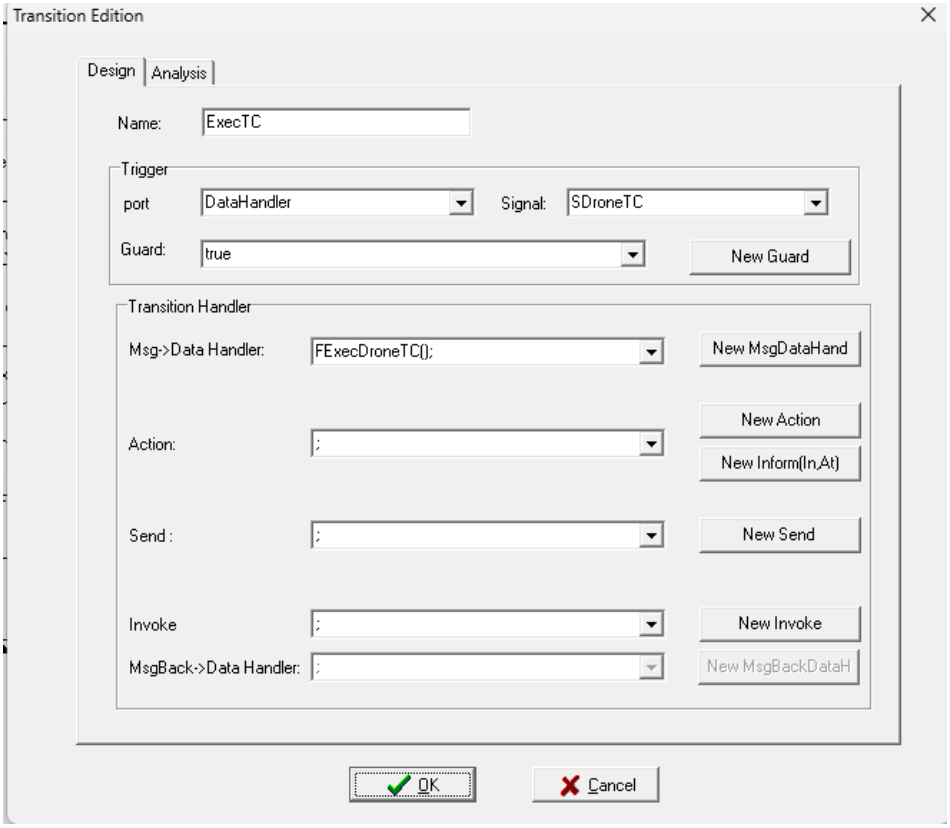
Init



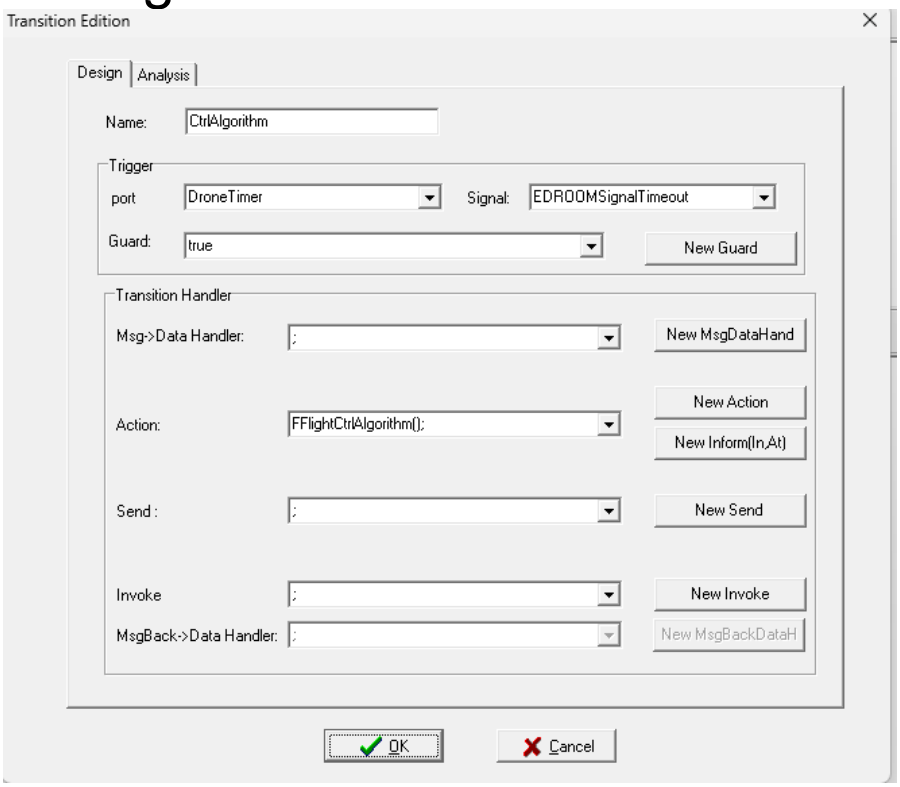
Transición1



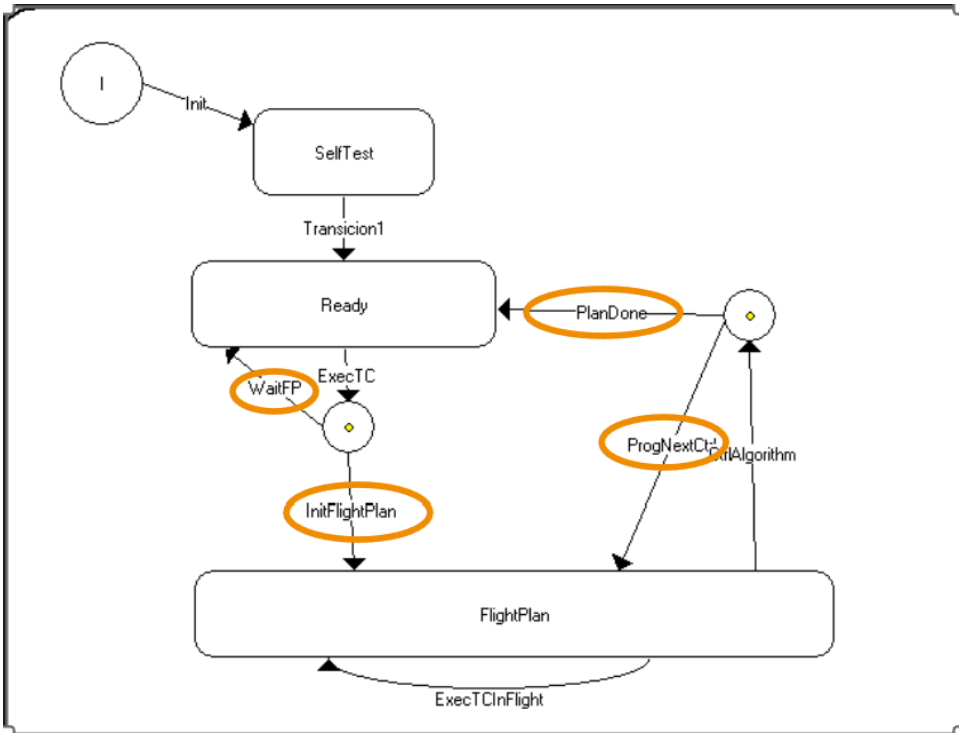
ExecTC



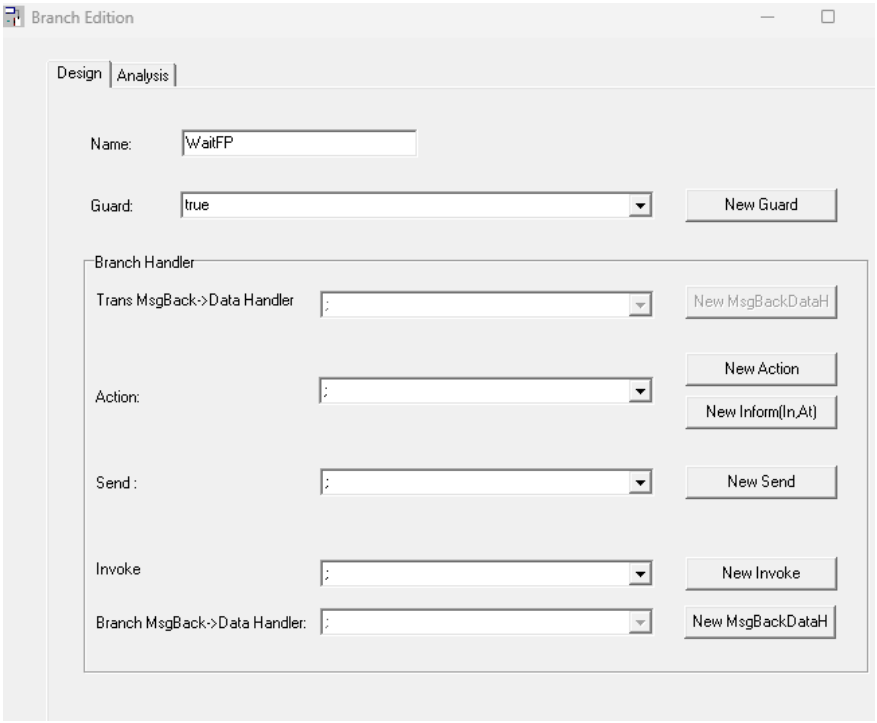
CtrlAlgorithm



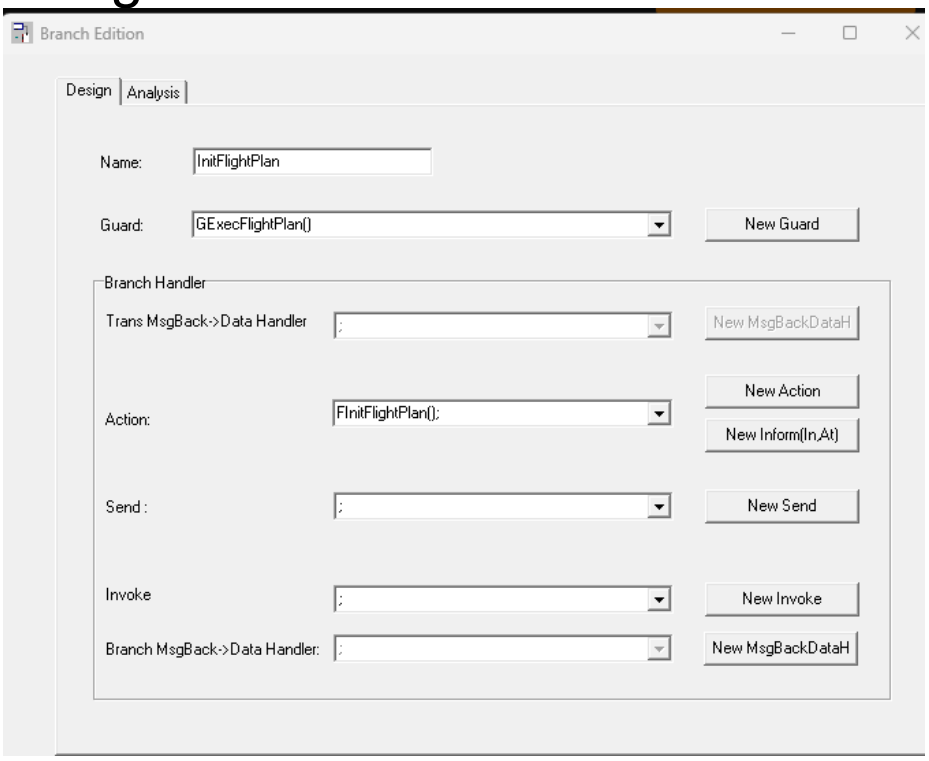
TODO 07 Añade la guarda y la acción asociada de cada una de las ramas de un choice points (utiliza el diagrama de estados varias veces si lo ves necesario)



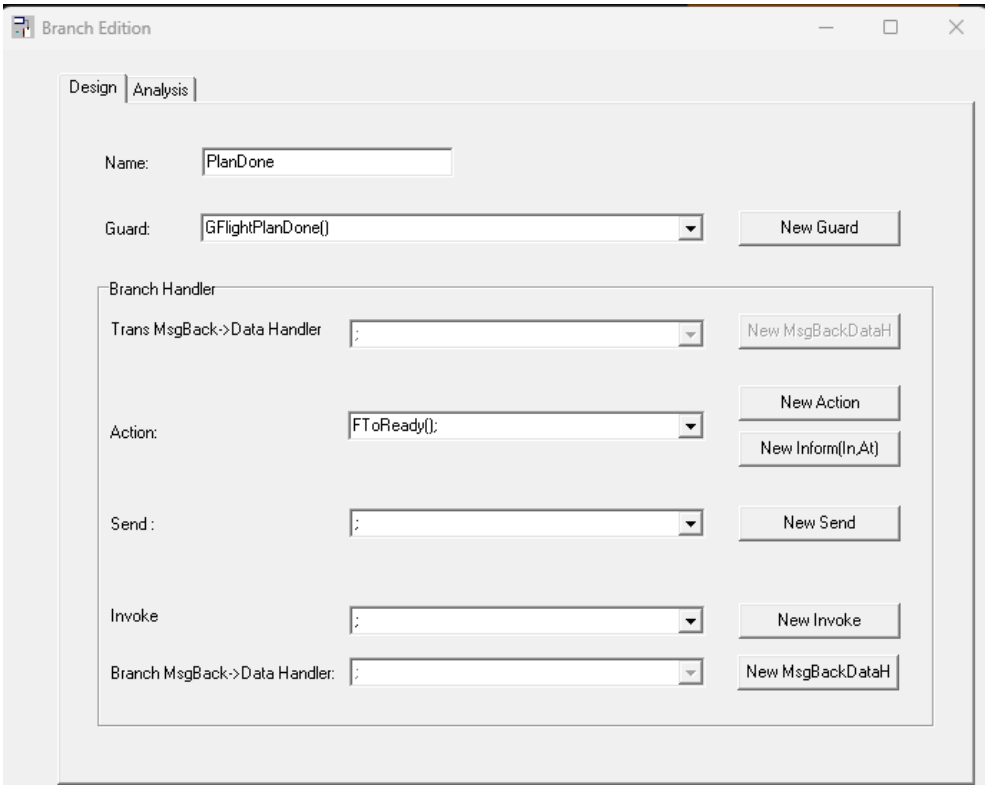
WaitFP



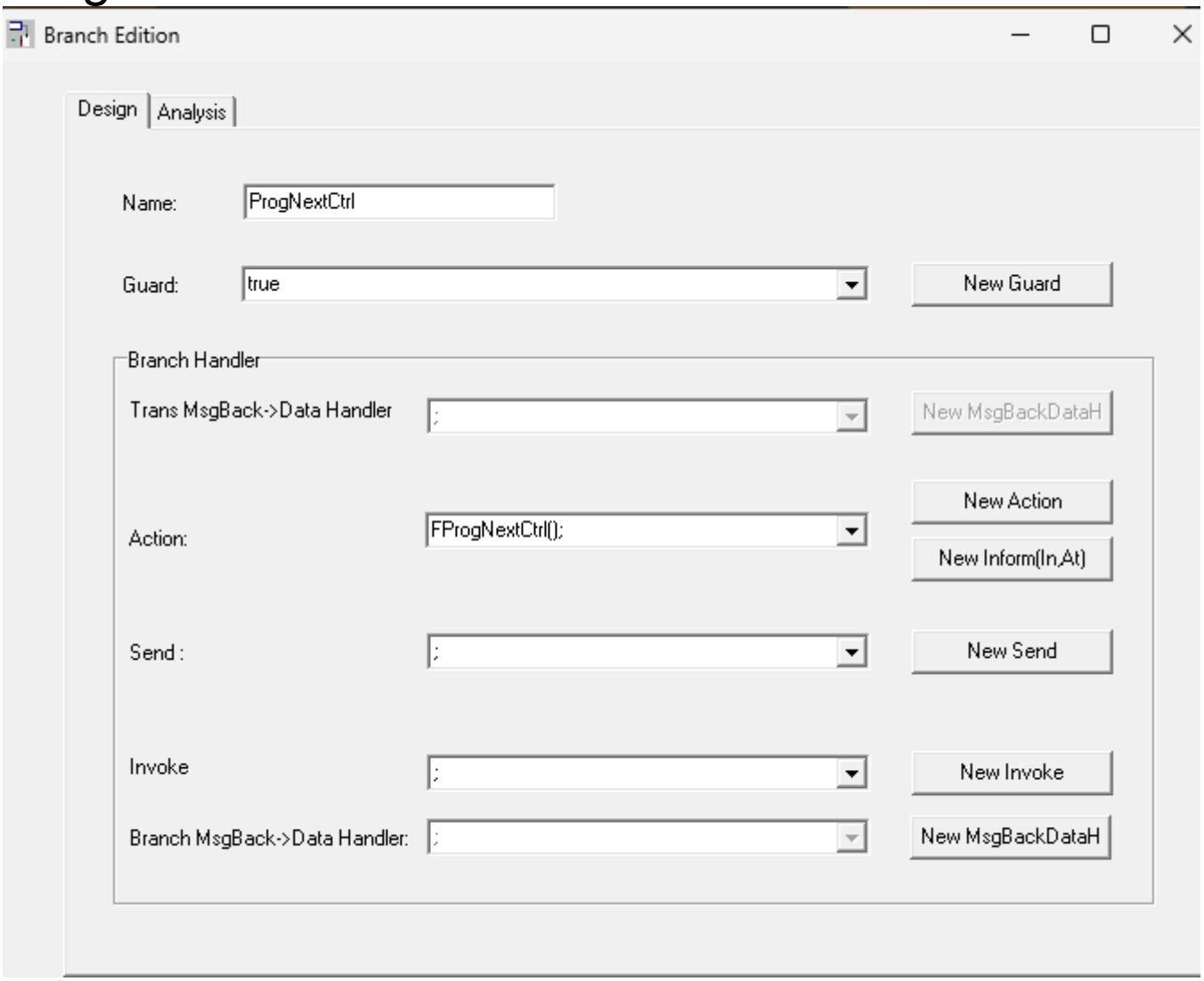
InFlightPlan



PlanDone



ProgNextCtrl



TODO 08 Añade el código de las acciones y guardas (marca en azul el código que se genera automáticamente al solicitar un servicio)

Acciones

FdroneSelfTest():

```
void FDroneSelfTest() {
 pus_service129_drone_selftest();
}
```

FGetConfig():

```
void FGetConfig() {
 CDDroneConfig varSDroneSetUp = *(CDDroneConfig*)Msg->data;
 pus_service129_setup(varSDroneSetUp);
}
```

FToReady():

```
void FToReady() {
 // transition action – pus service129 drone ready() called in FReplyDroneReady
}
```

FReplyDroneReady():

```
void FReplyDroneReady() {
 pus_service129_drone_ready();
 Msg->reply(SDroneReady);
}
```

FExecDroneTC():

```
void FExecDroneTc() {
 CCDTCHandler varSDroneTC = *(CCDTCHandler*)Msg->data;
 varSDroneTC.ExecDroneTC();
}
```

FInitFlightPlan():

```
void FInitFlightPlan() {
 Pr_Time time;
 time.GetTime(); // Get current monotonic time
 time += Pr_Time(0, 100000); // Add 100 ms
 pus_service129_init_flight_plan();
 DroneTimer.InformAt(time); // Program absolute timer
}
```

FProgNextCtrl():

```
void FProgNextCtrl() {
 Pr_Time time;
 time.GetTime();
 time += Pr_Time(0, 100000); // Advance by 100 ms
 time = VNextCtrl;
 DroneTimer.InformAt(time); // Re-program timer
}
```

FFlightCtrlAlgorithm():

```
void FFlightCtrlAlgorithm() {
 pus_service129_flight_ctrl_algorithm();
}
```

Guardas

GexecFlightPlan():

```
bool GExecFlightPlan() {
 return pus_service129_flight_plan_ready();
}
```

GExecFlightPlanDone():

```
bool GFlightPlanDone() {
 return pus_service129_flight_plan_done();
}
```